

Homework 02

Question 1: Training and test error under a fixed design

- a. Average training/test MSE vs. m ; verify training MSE is non-increasing.

```
library(ggplot2)
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
set.seed(43202)

n <- 100; p <- 20; n_sim <- 200
X_all <- matrix(rnorm(n * p), nrow = n, ncol = p)
beta <- 0.4^sqrt(1:p)
mu <- as.numeric(X_all %*% beta)

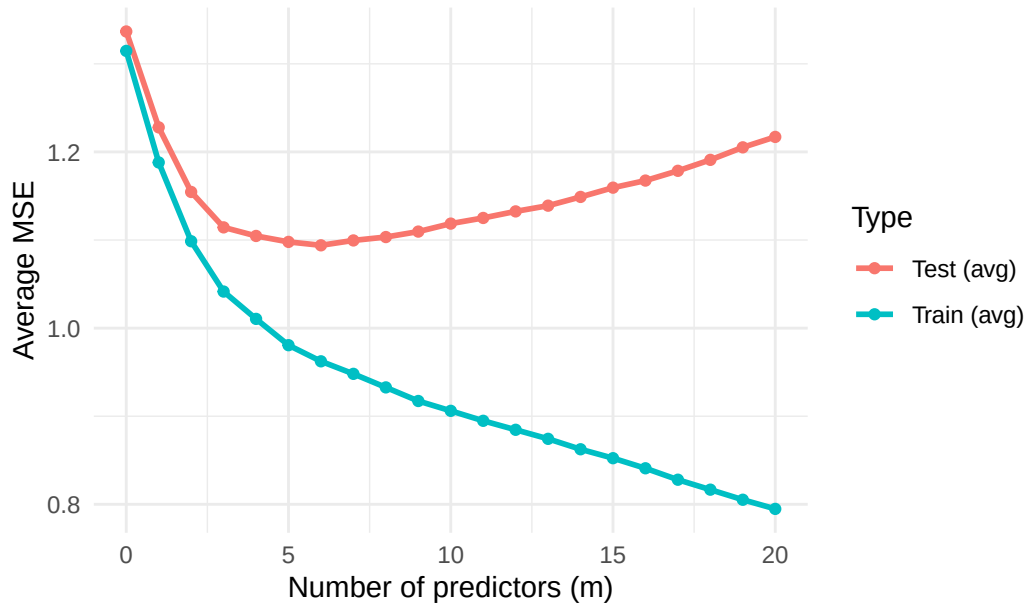
train_mse <- matrix(NA, n_sim, p + 1)
test_mse <- matrix(NA, n_sim, p + 1)

for (sim in 1:n_sim) {
  eps <- rnorm(n); eps_star <- rnorm(n)
  y <- mu + eps; y_star <- mu + eps_star
  for (m in 0:p) {
    if (m == 0) { fit <- lm(y ~ 1) } else {
      Xm <- X_all[, 1:m, drop = FALSE]; fit <- lm(y ~ Xm)
    }
    yhat_m <- fitted(fit)
    train_mse[sim, m + 1] <- mean((y - yhat_m)^2)
    test_mse[sim, m + 1] <- mean((y_star - yhat_m)^2)
  }
}

avg_train_mse <- colMeans(train_mse)
avg_test_mse <- colMeans(test_mse)
m_seq <- 0:p

df_a <- data.frame(m = rep(m_seq, 2), MSE = c(avg_train_mse, avg_test_mse),
                  Type = rep(c("Train (avg)", "Test (avg)"), each = p + 1))
ggplot(df_a, aes(m, MSE, color = Type)) + geom_line(linewidth = 1) + geom_point(size = 1.5) +
  labs(title = "Average Training and Test MSE vs. Number of Predictors",
       x = "Number of predictors (m)", y = "Average MSE") + theme_minimal()
```

Average Training and Test MSE vs. Number of Predictors



```
all(t(apply(train_mse, 1, diff)) <= 1e-8)
```

```
[1] TRUE
```

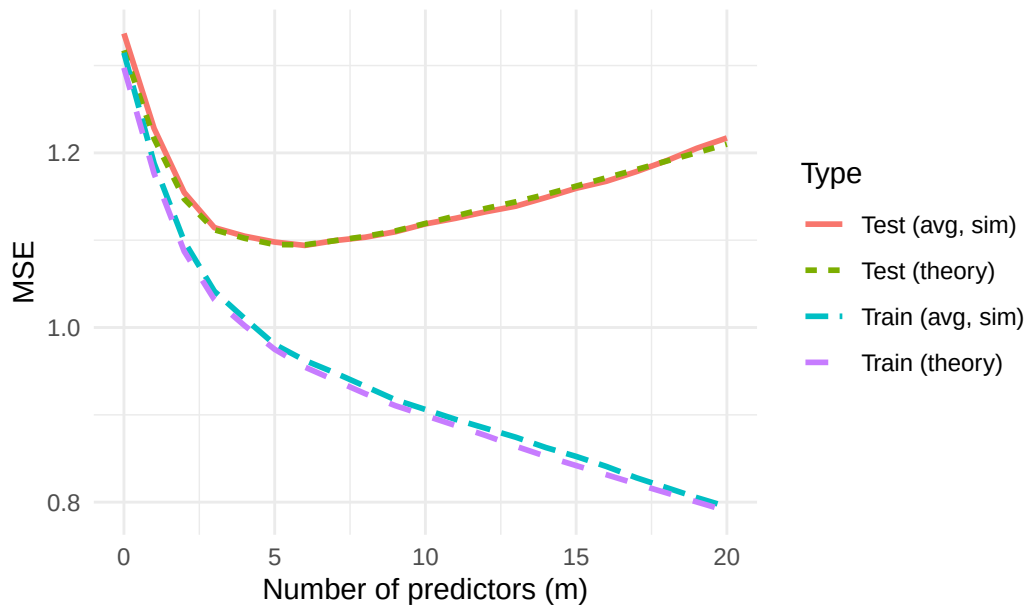
Training MSE never increases: a larger model nests the smaller one, so least squares can't do worse.

b. Add theoretical expectations to the plot.

```
B2 <- numeric(p + 1)
for (m in 0:p) {
  Xm <- if (m == 0) matrix(1, n, 1) else cbind(1, X_all[, 1:m, drop = FALSE])
  Hm <- Xm %*% solve(t(Xm) %*% Xm) %*% t(Xm)
  B2[m + 1] <- sum(((diag(n) - Hm) %*% mu)^2)
}
theory_train <- B2 / n + 1 - (m_seq + 1) / n
theory_test <- B2 / n + 1 + (m_seq + 1) / n
```

```
df_b <- data.frame(m = rep(m_seq, 4),
  MSE = c(avg_train_mse, avg_test_mse, theory_train, theory_test),
  Type = rep(c("Train (avg, sim)", "Test (avg, sim)", "Train (theory)", "Test (theory)"), each = p + 1))
ggplot(df_b, aes(m, MSE, color = Type, linetype = Type)) + geom_line(linewidth = 1) +
  labs(title = "Simulated vs. Theoretical Expected MSE", x = "Number of predictors (m)", y = "MSE") +
  theme_minimal()
```

Simulated vs. Theoretical Expected MSE

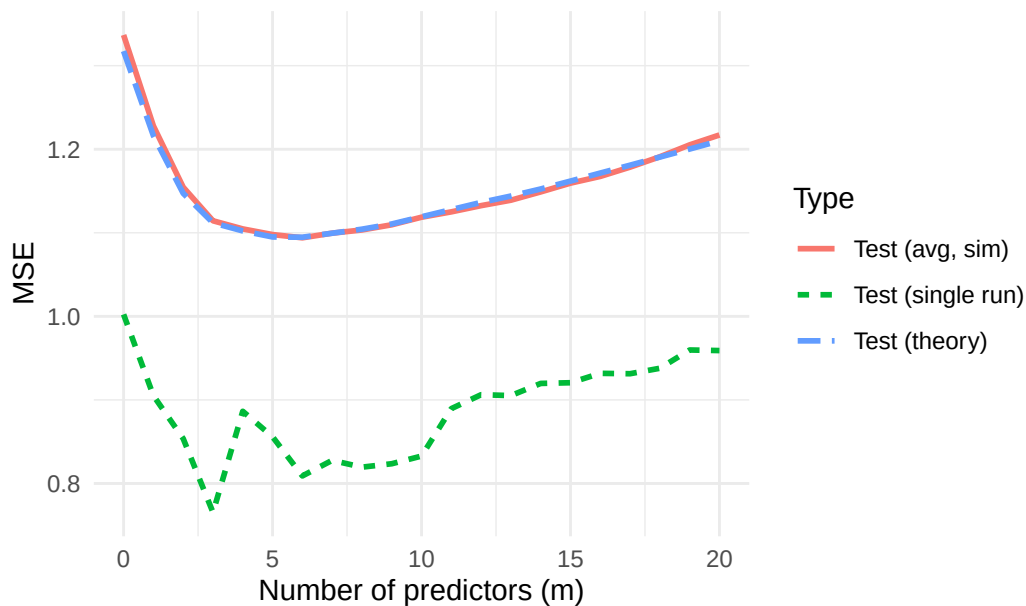


Theory and simulation nearly overlap; 200 replications keep Monte Carlo error small.

c. Add a single-run test MSE curve; explain smoothness and why training MSE can't select m .

```
df_c <- data.frame(m = rep(m_seq, 3), MSE = c(avg_test_mse, theory_test, test_mse[1, ]),
  Type = rep(c("Test (avg, sim)", "Test (theory)", "Test (single run)"), each = p + 1))
ggplot(df_c, aes(m, MSE, color = Type, linetype = Type)) + geom_line(linewidth = 1) +
  labs(title = "Average vs. Single-Run Test MSE", x = "Number of predictors (m)", y = "MSE") +
  theme_minimal()
```

Average vs. Single-Run Test MSE



A single run reflects one noisy draw of ϵ, ϵ^* ; averaging 200 runs cancels that noise.

Expected training and test MSE share the same bias term but opposite-signed variance: $E(\text{MSE}_{\text{train},m}) = B_m^2/n - (m+1)/n + 1$ vs. $E(\text{MSE}_{\text{test},m}) = B_m^2/n + (m+1)/n + 1$. Training MSE's falling bias and *subtracted*

variance term both push it down, so it never signals overfitting; test MSE's *added* $(m+1)/n$ penalizes extra estimated coefficients — hence the need for a validation/test set to choose m .

Question 2: Prediction error at one target point

```
n <- 100; p <- 6; sigma2 <- 1
X_all <- outer(1:n, 1:p, function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n))
beta <- rep(0.5, p); x0 <- c(0, 1, 0, 0, 1, 0)
mu0 <- sum(x0 * beta)
mu0
```

```
[1] 1
```

a. Theoretical bias and expected squared error

```
bias2 <- numeric(p + 1); var_term <- numeric(p + 1)
for (m in 0:p) {
  omitted_sum <- if (m == p) 0 else sum(x0[(m+1):p] * beta[(m+1):p])
  bias2[m + 1] <- omitted_sum^2
  included_sq <- if (m == 0) 0 else sum(x0[1:m]^2)
  var_term[m + 1] <- (sigma2 / n) * (1 + included_sq)
}
expected_sq_err <- bias2 + var_term
data.frame(m = 0:p, squared_bias = bias2, variance = var_term, expected_squared_error = expected_sq_err)
```

	m	squared_bias	variance	expected_squared_error
1	0	1.00	0.01	1.01
2	1	1.00	0.01	1.01
3	2	0.25	0.02	0.27
4	3	0.25	0.02	0.27
5	4	0.25	0.02	0.27
6	5	0.00	0.03	0.03
7	6	0.00	0.03	0.03

$\mu_0 = 1$ since only $x_{0,2} = x_{0,5} = 1$ are nonzero. Omitted predictors leave a fixed bias (signal they can't capture); included coefficients add estimation variance. Since $x_{0,j} = 0$ for $j = 1, 3, 4, 6$, error is flat across $m = 0 \rightarrow 1, 2 \rightarrow 4, 5 \rightarrow 6$, dropping only at $m = 2, 5$. Minimum is at $m = 5$.

b. Simulation, seed 43203

```
set.seed(43203)
n_sim <- 200
sq_err <- matrix(NA, n_sim, p + 1)
for (sim in 1:n_sim) {
  eps <- rnorm(n)
  y <- as.numeric(X_all %*% beta) + eps
  for (m in 0:p) {
    if (m == 0) { muhat <- unname(coef(lm(y ~ 1))[1]) } else {
      Xm <- X_all[, 1:m, drop = FALSE]; cf <- coef(lm(y ~ Xm))
      muhat <- unname(cf[1] + sum(cf[-1] * x0[1:m]))
    }
  }
}
```

```

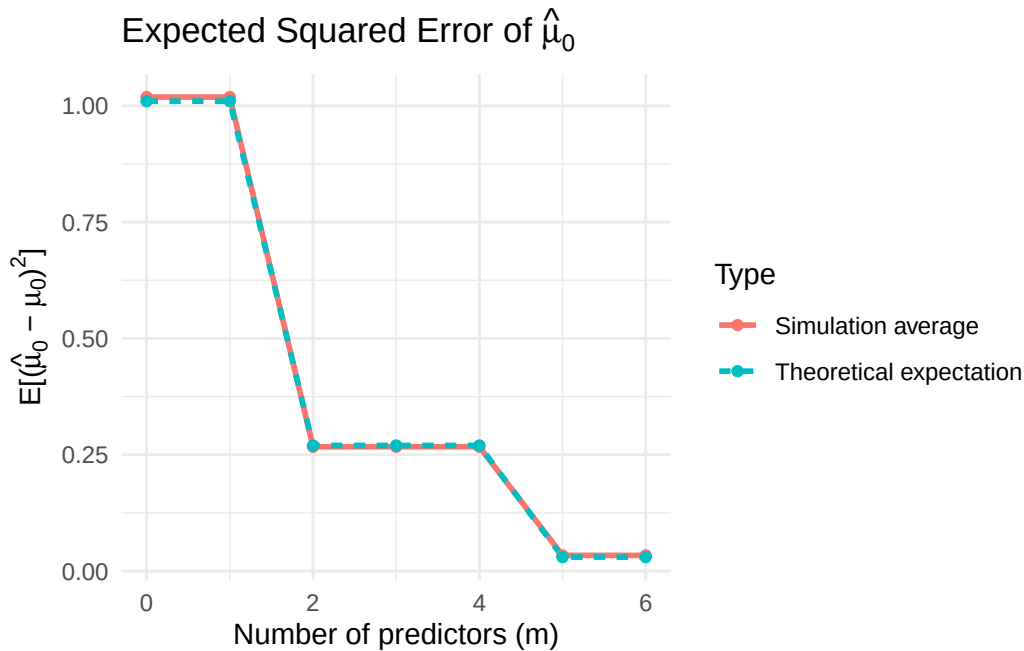
    sq_err[sim, m + 1] <- (muhat - mu0)^2
  }
}
avg_sq_err <- colMeans(sq_err)

```

```

library(ggplot2)
m_seq <- 0:p
df_q2 <- data.frame(m = rep(m_seq, 2), Error = c(avg_sq_err, expected_sq_err),
  Type = rep(c("Simulation average", "Theoretical expectation"), each = p + 1))
ggplot(df_q2, aes(m, Error, color = Type, linetype = Type)) + geom_line(linewidth = 1) + geom_point(size = 10) +
  labs(title = expression(paste("Expected Squared Error of ", hat(mu)[0])),
    x = "Number of predictors (m)", y = expression(paste("E[(", hat(mu)[0], " - ", mu[0], ")^2, "]"))) +
  theme_minimal()

```



```

best_m <- m_seq[which.min(expected_sq_err)]
best_m

```

```
[1] 5
```

Simulation tracks theory closely. Smallest minimizing $m = 5$.

c. Why only predictors 2 and 5 matter here

$\mu_0 = \sum_j x_{0j}\beta_j$ picks up only $j = 2, 5$, regardless of other β_j . Question 1's test MSE averages error over *all* rows, so every nonzero β_j matters somewhere; a single-point target is sensitive only to coefficients aligned with its nonzero coordinates — overall and target-specific error can favor different m .

Question 3: The optimism correction

a. Derivation

Since $\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}$, expanding $n \cdot \text{MSE}_{\text{train}}$ and taking $E(\cdot \mid \mathbf{X})$: the cross term vanishes, the first term is B^2 , and $E(\boldsymbol{\epsilon}^T(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}) = \sigma^2(n - (p + 1))$, giving $E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 - (p + 1)/n)$.

Similarly $\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}$; cross terms vanish, $E\|\boldsymbol{\epsilon}^*\|^2 = \sigma^2 n$, $E(\boldsymbol{\epsilon}^\top \mathbf{H}\boldsymbol{\epsilon}) = \sigma^2(p+1)$, giving $E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 + (p+1)/n)$.

b. Expected optimism

Bias terms cancel: $E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = 2\sigma^2(p+1)/n$.

```
n <- 120; p <- 4; B2_over_n <- 0.04; sigma2 <- 1
E_train <- B2_over_n + sigma2 * (1 - (p+1)/n)
E_test  <- B2_over_n + sigma2 * (1 + (p+1)/n)
c(E_train = E_train, E_test = E_test, optimism = E_test - E_train)
```

```
      E_train      E_test      optimism
0.99833333 1.08166667 0.08333333
```

These are expectations over repeated noise draws with $\mathbf{X}$ fixed, not a per-dataset guarantee — an unusual draw can flip the realized ordering even though training MSE is optimistic on average.

c. C_p and the corrected test MSE

```
RSS <- 116.4; sigma2_hat <- 0.96
MSE_test_hat <- (RSS + 2*(p+1)*sigma2_hat) / n
Cp <- RSS/sigma2_hat - n + 2*(p+1)
c(MSE_test_hat = MSE_test_hat, Cp = Cp)
```

```
MSE_test_hat      Cp
      1.05      11.25
```

Solving C_p for RSS gives $\widehat{\text{MSE}}_{\text{test}} = \hat{\sigma}^2(1 + C_p/n)$, confirmed numerically. Since $\hat{\sigma}^2/n$ is a fixed constant shared across candidates, $\widehat{\text{MSE}}_{\text{test}}$ is increasing linear in C_p , so minimizing one minimizes the other.

Question 4: Comparing Mallows' C_p , AIC, and BIC

```
diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]; n <- 370

modelA_vars <- c("bmi","bp","s5","sex","s1","s2","s4")
modelB_vars <- c(modelA_vars, "s6")
full_vars   <- c("age","sex","bmi","bp","s1","s2","s3","s4","s5","s6")

fitA <- lm(y ~ ., data = train[, c(modelA_vars, "y")])
fitB <- lm(y ~ ., data = train[, c(modelB_vars, "y")])
fitFull <- lm(y ~ ., data = train[, c(full_vars, "y")])

RSS_A <- sum(resid(fitA)^2); RSS_B <- sum(resid(fitB)^2); RSS_full <- sum(resid(fitFull)^2)
p_A <- 7; p_B <- 8; p_full <- 10
dof_full <- n - (p_full + 1)
sigma2_hat <- RSS_full / dof_full
data.frame(Model = c("A","B","Full"), p = c(p_A,p_B,p_full), RSS = c(RSS_A,RSS_B,RSS_full))
```

	Model	p	RSS
1	A	7	1098909
2	B	8	1089717
3	Full	10	1089452

```
c(dof_full = dof_full, sigma2_hat = sigma2_hat)
```

```
dof_full sigma2_hat
359.000    3034.684
```

a. $\hat{\sigma}^2 \approx 3034.68$ using $\text{RSS}_{\text{full}}/359$.

b.

```
Cp_fun <- function(rss, p, n, s2) rss/s2 - n + 2*(p+1)
AICred_fun <- function(rss, p, n) n*log(rss/n) + 2*(p+1)
BICred_fun <- function(rss, p, n) n*log(rss/n) + (p+1)*log(n)

data.frame(Model = c("A","B"), p = c(p_A,p_B),
  Cp = c(Cp_fun(RSS_A,p_A,n,sigma2_hat), Cp_fun(RSS_B,p_B,n,sigma2_hat)),
  AIC_red = c(AICred_fun(RSS_A,p_A,n), AICred_fun(RSS_B,p_B,n)),
  BIC_red = c(BICred_fun(RSS_A,p_A,n), BICred_fun(RSS_B,p_B,n)))
```

	Model	p	Cp	AIC_red	BIC_red
1	A	7	8.116253	2974.640	3005.948
2	B	8	7.087434	2973.532	3008.754

C_p and AIC select **Model B**; BIC selects **Model A**.

c. s_6 drops RSS by ≈ 9192 . C_p /AIC charge a fixed penalty ($2\hat{\sigma}^2 \approx 6069$), which the drop clears, favoring B. BIC's penalty is $\hat{\sigma}^2 \log(n) \approx 17,947$ (since $\log(370) \approx 5.91 > 2$), which the drop doesn't clear, favoring A — BIC's usual conservatism as n grows.

Question 5: Validation and final test data

a.

```
train <- diabetes[1:370, ]; test <- diabetes[371:442, ]
predictor_order <- c("age","sex","bmi","bp","s1","s2","s3","s4","s5","s6")

fit_and_predict <- function(m, train, newdata) {
  if (m == 0) { fit <- lm(y ~ 1, data = train) } else {
    vars <- predictor_order[1:m]; fit <- lm(y ~ ., data = train[, c(vars, "y")])
  }
  predict(fit, newdata = newdata)
}

test_mse <- sapply(0:10, function(m) mean((test$y - fit_and_predict(m, train, test))^2))
data.frame(m = 0:10, test_MSE = test_mse)
```

	m	test_MSE
1	0	5822.890
2	1	5527.791
3	2	5527.210

```

4  3 3719.235
5  4 3246.823
6  5 3234.919
7  6 3207.615
8  7 2636.502
9  8 2624.924
10 9 2369.344
11 10 2484.776

```

```
c(best_m = (0:10)[which.min(test_mse)], min_mse = min(test_mse))
```

```

best_m  min_mse
9.000 2369.344

```

The misuse is step 3, not step 2: one-time evaluation is unbiased, but *selecting* by the smallest of eleven noisy estimates makes the test set part of model selection, biasing the reported minimum downward relative to the true error of the selected model.

b. Five-fold CV within training data

```

set.seed(43204)
fold_id <- sample(rep(1:5, length.out = nrow(train)))
cv_mse <- matrix(NA, 5, 11)
for (k in 1:5) {
  tr_fold <- train[fold_id != k, ]; va_fold <- train[fold_id == k, ]
  cv_mse[k, ] <- sapply(0:10, function(m) mean((va_fold$y - fit_and_predict(m, tr_fold, va_fold))^2))
}
avg_cv_mse <- colMeans(cv_mse)
data.frame(m = 0:10, CV_MSE = avg_cv_mse)

```

```

   m  CV_MSE
1  0 5988.423
2  1 5804.711
3  2 5841.856
4  3 3974.053
5  4 3734.873
6  5 3751.078
7  6 3796.594
8  7 3266.723
9  8 3283.287
10 9 3153.751
11 10 3133.770

```

```

best_m_cv <- (0:10)[which.min(avg_cv_mse)]
best_m_cv

```

```
[1] 10
```

```

final_preds <- fit_and_predict(best_m_cv, train, test)
final_test_mse <- mean((test$y - final_preds)^2)
final_test_mse

```

```
[1] 2484.776
```

Select m by CV within training data ($m = 10$ here); refit once on all 370 rows; only then evaluate once on the 72-row test set — never used for selection, so its MSE is an honest estimate.

c. Given the test set was already examined

Report the eleven values transparently, labeled as already used for selection, plus a training-only CV estimate, noting the reported minimum is likely optimistic. An honest re-evaluation needs a genuinely new, independent dataset — reusing the same 72 rows can't restore validity.